

PROJECT SPECIFICATION

CyberGuard AI

AI-Powered Home Network Security Monitor for IoT Devices

1. Business Problem Statement

The rapid adoption of Internet of Things (IoT) devices in homes has introduced significant security risks that most homeowners are unprepared to handle. Smart TVs, security cameras, thermostats, smart bulbs, voice assistants, and home NAS servers are now common household devices - yet each one represents a potential entry point for malicious actors.

Unlike corporate environments, where dedicated Security Operations Center (SOC) teams monitor network activity around the clock using enterprise-grade tools, homeowners have no equivalent protection. Commercial security platforms such as Darktrace and CrowdStrike are designed for enterprise networks and cost thousands of dollars per month - well beyond the reach of a typical household.

The specific problem this project targets is:

“How can a non-technical homeowner be alerted when their IoT devices are behaving suspiciously, and understand what to do about it - without needing any technical expertise?”

This problem is:

- Widespread - tens of millions of households now own multiple IoT devices
- Underserved - no affordable, user-friendly AI security tool exists for home networks
- High-impact - compromised IoT devices can expose home video footage, banking sessions, and personal data.

1.1 Target User:

Attribute	Description
Who	Non-technical homeowner with IoT devices
What they have	Smart TV, security camera, thermostat, smart bulbs, home NAS
What they lack	Technical knowledge, enterprise security tools, budget
What they need	Simple alerts in plain English and clear actions to take

2. Tool Concept and Scope

2.1 Primary Job-To-Be-Done

"This tool exists to help a non-technical homeowner detect unusual activity on their home network, understand what it means in plain English, and decide what action to take - by combining an unsupervised anomaly detection model with a local LLM-powered assistant"

2.2 Version 1 - Proof of Concept (Core Scope)

The following features define the first version of the system:

1. User login and signup - homeowners create a private account to access their dashboard
2. Network log upload - homeowner uploads a CSV file of their home network activity
3. Anomaly detection - an Isolation Forest model processes the uploaded data and flags unusual device behavior
4. Results page - flagged events displayed in plain, non-technical language with color-coded risk indicators (normal vs suspicious)
5. LLM explanation - homeowner can ask "What does this mean?" for any flagged event and receive a plain English explanation and suggested action
6. Basic database - stores user accounts, scan history, and flagged events.

2.3 Version 2 - Expanded Features (After V1 is Stable)

7. Per-device dashboard with Chart.js visualizations showing activity trends over time.
8. Device history tracking - the system remembers individual devices across multiple scans.
9. Case management - homeowner can mark flagged events as checked, safe, or confirmed problem, with notes.
10. Scan history - view and compare results across past scans.
11. Actionable LLM recommendations - specific steps such as changing a device password or checking for firmware updates.

2.4 Out-of-Scope

The following are explicitly excluded from this project:

- Real-time live network monitoring - the system analyzes uploaded log files, not live traffic.
 - Integration with physical network hardware or router APIs.
 - Mobile application development.
 - Cloud deployment or cloud-based inference.
 - Multi-agent orchestration or distributed architecture.
 - Automated remediation - the system only recommends actions, never takes them automatically.
-

3. Architecture Design Decisions

3.1 Three-Layer Design

The system follows a three-layer architecture:

Layer 1 - Anomaly Detection Engine (Isolation Forest)

Handles all anomaly detection tasks. Trained on the NSL-KDD network intrusion dataset. Flags network activity that deviates significantly from learned normal patterns. This layer is fully deterministic and does not involve the LLM. Uses an unsupervised approach - the model learns what normal looks like without being told what attacks look like in advance.

Layer 2 - Explainability Layer (SHAP)

Explains why each flagged event was considered anomalous by identifying the specific features that contributed most to the anomaly score. Outputs are translated into plain English descriptions before being shown to the homeowner.

Layer 3 - LLM Assistant (Local LLM via Ollama)

Handles all user interaction. The homeowner can ask free-form questions about any flagged event. The LLM receives only structured outputs from Layer 1 and Layer 2 - the anomaly score, risk category, and top contributing features. It does not have access to raw network data. It does not make decisions - it only interprets and explains. This mirrors the same disciplined pattern used in Project 1.

3.2 Data Flow

- Homeowner uploads CSV network log file
- Flask backend receives and validates the file
- Isolation Forest model processes the data and generates anomaly scores
- SHAP layer identifies top contributing features per flagged event
- Plain English translation layer converts technical feature names to homeowner-friendly language
- Results stored in SQL database and displayed on the results page
- Homeowner asks a question about a flagged event
- LLM receives structured context (anomaly score, risk category, SHAP factors) and generates a response

3.3 Why Isolation Forest Over Supervised Classification?

Supervised classification models (such as XGBoost, used in Project 1) require labeled training data - examples explicitly marked as normal or attack. In the real world, most home network traffic is unlabeled. No homeowner has a dataset of their own traffic with attack labels.

Isolation Forest is an unsupervised anomaly detection algorithm. It learns what normal traffic looks like and flags deviations without ever being shown labeled attack examples. This is a more realistic approach for home network security and introduces a meaningfully different ML technique from Project 1.

3.4 Why a Local LLM Over a Cloud API?

Home network logs contain sensitive information about device activity, usage patterns, and potentially identifiable behavior. Sending this data to a cloud API would create a privacy risk. A local LLM running via Ollama processes all data on the homeowner's own machine, with no data leaving the device.

3.5 Technology Stack

Component	Technology	Reason
Backend Framework	Flask	Lightweight Python web framework; enables full backend control vs Streamlit
Database	SQLite + SQLAlchemy	Simple local database; SQLAlchemy enables clean Python-to-SQL interaction
ML Model	Isolation Forest	Unsupervised anomaly detection; no labeled attack data required
Explainability	SHAP	Industry standard for explainability; used to identify top contributing features for each flagged anomaly score
Local LLM	Ollama + Phi-3 Mini	Runs entirely on CPU; no data sent externally; privacy preserving
Web Pages	HTML + Jinja2	Flask-native templating; enables dynamic data in web pages
Styling	Custom CSS	Professional dashboard design; full control over visual appearance
Interactivity	JavaScript + Chart.js	Live search, AJAX updates, and dashboard charts without page reloads
Dataset	NSL-KDD	Standard benchmark for network intrusion detection; 125,000+ records
Testing	pytest	Automated unit testing for all core functions
Version Control	Git + GitHub	Code management and submission to professor

4. Database Design

The system uses a relational SQL database with four tables:

4.1 Table: users

Column	Type	Description
id	Integer (PK)	Unique user identifier

Column	Type	Description
name	String	Homeowner full name
email	String (unique)	Login email address
password_hash	String	Securely hashed password
created_at	DateTime	Account creation timestamp

4.2 Table: scans

Column	Type	Description
id	Integer (PK)	Unique scan identifier
user_id	Integer (FK)	References users.id
filename	String	Original uploaded filename
total_events	Integer	Total records in uploaded file
flagged_count	Integer	Number of anomalies detected
scanned_at	DateTime	Timestamp of scan

4.3 Table: flagged_events

Column	Type	Description
id	Integer (PK)	Unique event identifier
scan_id	Integer (FK)	References scans.id
anomaly_score	Float	Model-generated anomaly score
risk_category	String	LOW, MEDIUM, or HIGH
plain_english	String	Human-readable description of anomaly
top_factors	String	JSON-encoded SHAP top features
flagged_at	DateTime	Timestamp of event

4.4 Table: llm_responses

Column	Type	Description
id	Integer (PK)	Unique response identifier
event_id	Integer (FK)	References flagged_events.id
question	String	Homeowner's question
response	String	LLM-generated answer
asked_at	DateTime	Timestamp of question

5. Functional Requirements

The following statements define the required behaviors of the system. Each statement is independently verifiable.

1. The system shall allow a homeowner to create an account with a name, email, and password.
 2. The system shall authenticate users via email and password before granting access to any features.
 3. The system shall accept a CSV file upload containing home network activity log data.
 4. The system shall validate the uploaded file against a predefined schema and reject invalid or incomplete files with a clear error message.
 5. The system shall preprocess uploaded network data using deterministic cleaning rules before passing it to the ML model.
 6. The system shall apply a trained Isolation Forest model to the uploaded data and generate an anomaly score for each record.
 7. The system shall classify each record into a risk category (LOW, MEDIUM, HIGH) based on its anomaly score.
 8. The system shall use SHAP to identify the top contributing features for each flagged event.
 9. The system shall translate technical feature names and values into plain English descriptions accessible to a non-technical homeowner.
 10. The system shall display flagged events on a results page with color-coded risk indicators (green for normal, red for suspicious).
 11. The system shall store scan results and flagged events in a SQL database associated with the logged-in user.
 12. The system shall provide a conversational Q&A interface where the homeowner can ask questions about any flagged event.
 13. The system shall ground all LLM responses in the anomaly score, risk category, and SHAP factors for the specific event - not general knowledge.
 14. The system shall handle an unknown or invalid CSV file gracefully by returning a clear error message without crashing.
 15. The system shall operate fully offline after initial setup, with no data transmitted to external servers.
-

6. Non-Functional Requirements

6.1 Privacy

- All network log data shall be processed locally on the user's machine.
- No data shall be transmitted to external APIs or cloud services.
- User passwords shall be stored as hashed values, never in plain text.

6.2 Offline Operation

- The system shall operate fully offline after model training and LLM download.
- All inference shall run locally using Ollama.

6.3 Reproducibility

- ML model results shall be fully reproducible given the same trained model and input data, achieved by setting a fixed random seed during training.

6.4 Failure Modes

- The system shall validate CSV schema on upload and reject files with missing required columns with a descriptive error message.
 - The system shall handle missing or null values in uploaded data using predefined fallback rules without crashing.
 - The system shall return a controlled error message if the LLM fails to respond.
 - The system shall require authentication before accessing any feature; unauthenticated requests shall redirect to the login page.
-

7. Interface Requirements

7.1 Web Pages

Page	Description
Login / Signup	Homeowner creates account or logs in
Dashboard / Home	Welcome page showing recent scan summary
Upload Scan	Drag and drop or select CSV file for upload
Scan Results	Table of flagged events with color-coded risk badges
Event Detail	Full anomaly explanation, SHAP factors, and LLM Q&A box

7.2 JavaScript Interactions

- Live search and filter on the scan results table
- Color-coded risk badges that update dynamically
- AJAX-based LLM Q&A - homeowner submits a question and receives a response without a full page reload

7.3 Example Output (Event Detail Page)

Device: Unknown Device (192.168.1.45)

Anomaly Score: 0.84 - HIGH RISK

What happened: This device sent an unusually large amount of data to an unfamiliar address late at night. This type of behavior can sometimes indicate the device has been compromised.

Top factors that triggered this alert:

- Data sent was much larger than usual
- Connection happened between 2am and 4am
- Destination address has not been seen before

Homeowner question: Should I be worried about this?

AI Response: Based on the analysis, this event has characteristics that are worth investigating. The combination of an unusual time, unfamiliar destination, and high data volume is not typical for normal device behavior. I would recommend disconnecting this device from your network temporarily and checking whether a firmware update is available.

8. Data Requirements

8.1 Training Dataset

The system shall use the NSL-KDD network intrusion detection dataset for training the Isolation Forest model.

Attribute	Value
Dataset Name	NSL-KDD
Training Records	125,973
Test Records	22,544
Features	41 network connection attributes
Labels	Normal or one of 4 attack categories (DoS, Probe, R2L, U2R)
Training Approach	Unsupervised - labels used only for post-hoc validation
Source	Publicly available for research use

8.2 Key Features

- duration - length of the network connection in seconds
- protocol_type - type of protocol used (TCP, UDP, ICMP)
- service - network service on the destination (http, ftp, smtp, etc.)
- src_bytes - number of data bytes sent from source to destination
- dst_bytes - number of data bytes sent from destination to source
- flag - normal or error status of the connection
- count - number of connections to the same host in the past 2 seconds
- srv_count - number of connections to the same service in the past 2 seconds

8.3 Local Storage Policy

- The training dataset shall be stored locally in the /data directory.
 - Uploaded scan files shall be processed in memory and not persisted to disk.
 - The trained model shall be saved to /outputs/model.pkl for reuse.
 - Session logs shall be written to /outputs/session.log.
-

9. Evaluation Plan

9.1 Model Evaluation

Since the system uses unsupervised learning, traditional accuracy metrics do not directly apply during training. The following approach will be used:

- Train the Isolation Forest model on NSL-KDD data WITHOUT using the attack labels
- Evaluate the model's flagging behavior AGAINST the known labels to measure how well the unsupervised approach detects real attacks
- Report: Precision, Recall, and F1-score of anomaly detection against known attack records
- Report: Anomaly score distribution across normal vs attack records

9.2 LLM Response Quality Rubric

Each LLM response to a homeowner question is scored on a 0-3 scale:

Score	Criteria
3	Response directly answers the question, cites specific anomaly data, contains no fabricated information, uses plain non-technical language
2	Response answers the question but is partially technical, vague, or only loosely references the anomaly data
1	Response is generic or only partially answers the question
0	Response is incorrect, hallucinated, or completely misses the question

9.3 Test Scenarios

Scenario 1 - Normal Traffic File

Input: CSV file containing only normal network activity. Expected: No events flagged, dashboard shows all green.

Scenario 2 - File with Known Attack Patterns

Input: CSV containing records matching known attack patterns from NSL-KDD. Expected: Suspicious events flagged with HIGH risk, correct SHAP factors identified.

Scenario 3 - Unknown User ID Access Attempt

Input: Unauthenticated request to access dashboard. Expected: Redirect to login page, no data exposed.

Scenario 4 - Invalid CSV Schema

Input: CSV missing required columns. Expected: System rejects file with clear error message, no partial processing.

Scenario 5 - LLM Q&A Grounding Check

Input: Homeowner asks about a flagged event. Expected: LLM response references actual anomaly score and SHAP factors, no fabricated information.

Scenario 6 - Empty CSV File

Input: CSV file with headers but no data rows. Expected: System handles gracefully with a clear message, no crash.

10. Ten-Week Implementation Timeline

Week	Focus	Deliverables	Definition of Done
1	Planning + Specification	GitHub repo, folder structure, project specification, dataset download	Spec submitted to professor; repo structure clean and pushed
2	Flask Foundation	Flask app running, database tables created, user signup and login working	A homeowner can create an account and log in successfully
3	File Upload + Preprocessing	CSV upload working, data validation, preprocessing pipeline	Invalid files rejected with error; valid files processed without crash
4	ML Model Training	Isolation Forest trained on NSL-KDD, model saved, anomaly scores generated	Model produces anomaly scores; validated against known labels
5	SHAP + Plain English Layer	SHAP values calculated, plain English translation working	Each flagged event shows top factors in homeowner-friendly language
6	Results Page + Database	Results stored in DB, results page displays flagged events with color coding	Homeowner sees scan results with risk badges after uploading file
7	LLM Q&A Integration	Ollama connected, Q&A working on event detail page	Homeowner can ask a question and receive a grounded plain English answer
8	CSS + JavaScript Polish	Dashboard styled, live search, AJAX Q&A, Chart.js (V2 prep)	Application looks professional; interactions work without page reloads
9	Testing + Bug Fixes	pytest unit tests, all 6 test scenarios executed and documented	All tests pass; edge cases handled without crash
10	Final Report + Submission	Final report (2000+ words), README, clean GitHub repo, email to professor	Repo is public, organized, and complete with all deliverables

11. Skills and Technology Learning Map

Weeks 1-2 - Foundation Skills

Skill	Tool	
Flask routing and views	Flask	Build a route that returns Hello World; then a route that accepts a form submission

Skill	Tool	
Database design and ORM	SQLAlchemy	Define the 4 database tables; insert a test user; query it back
User authentication	Flask-Login, Werkzeug	Build signup, login, and logout; protect a route so only logged-in users can access it
HTML templating	Jinja2	Build a base template with navigation; extend it for login and dashboard pages
Version control	Git, GitHub	Commit after every working feature; never commit broken code

Weeks 3-5 - ML and Data Skills

Skill	Tool	Practice Task
Data loading and cleaning	pandas	Load NSL-KDD; handle missing values; encode categorical columns
Unsupervised anomaly detection	scikit-learn IsolationForest	Train model on NSL-KDD normal records; generate anomaly scores; validate against labels
Model saving and loading	pickle	Save trained model to disk; load it for inference without retraining
SHAP explainability	SHAP TreeExplainer	Generate SHAP values for 5 flagged events; print top 3 factors each
Plain English translation	Python string formatting	Map technical feature names to homeowner-friendly descriptions

Weeks 6-8 - Web and Integration Skills

Skill	Tool	Practice Task
File upload handling	Flask, pandas	Accept CSV upload; validate schema; pass to ML model; store results in DB
LLM integration	Ollama, Python	Send structured context to local LLM; receive and display grounded response
CSS dashboard styling	Custom CSS	Style the results page with color-coded risk badges and clean layout
JavaScript interactivity	Vanilla JS	Add live search filter on results table; AJAX form for LLM Q&A
Exception handling	Python try/except	Handle invalid CSV, missing columns, unknown errors without crashing

Weeks 9-10 - Evaluation and Reporting Skills

Skill	Tool	Practice Task
Unit testing	pytest	Write tests for model loading, anomaly scoring, CSV validation, and DB operations
LLM evaluation	Manual rubric (0-3)	Score 5 LLM responses; record results; identify any grounding issues
Technical writing	Markdown, Word	Write final report covering design decisions, results, limitations, lessons learned
Repository hygiene	Git, .gitignore	Ensure no model files, cache files, or temp files are committed